江蘇大學

Jiangsu University

Design of Casting Data Visualization Based on Python

Design of Casting Data Visualization Based on Python

School Name: School of Materials Science and Engineering

Major and Class: Metal Forming 1603

Student Name: Yang Ye

Supervisor's Name: Zhao Guoping

Supervisor's Title: Associate Professor

June 2020

Contents

List of Figures and Tables.
List of Figures
List of Tables.
List of Code Segments
Introduction
Chapter 1 Industry 4.0 and Intelligent Manufacturing
1.1 Theory of Industry 4.0
1.2 Digital Transformation of Manufacturing Abroad
1.3 Domestic Manufacturing in the Industry 4.0 Era
1.4 The Casting Industry in the Context of Intelligent Manufacturing
Chapter 2 Application of Information Technology in the Casting Industry
2.1 Real-Time Production Management
2.2 Intelligent Casting Systems
2.3 Casting Data Modeling
Chapter 3 Features of Python
3.1 Wide Range of Applications.
3.2 Suitability for Data Processing.
3.2.1 Scientific Computing
3.2.2 Enterprise Operations and Maintenance
3.2.3 Data Visualization.
3.3 Suitability for Artificial Intelligence
Chapter 4 Python and Industry 4.0.
4.1 Python and the Internet of Things
4.1.1 Building an IoT Platform
4.1.2 MicroPython.
4.1.3 Industrial Control Information Management
4.2 Building a Data Analysis Platform
4.3 Process Simulation.
Chapter 5 Initial Setup for Data Processing
5.1 Initial Setup
5.2 Basic Information of the Data
5.3 Introduction and Import of Extension Libraries
5.3.1 Brief Introduction to the Libraries.
5.3.2 Import of Data and Libraries
Chapter 6 Determination of Data Correlation
6.1 Removal of Data Outliers
6.2 Visualization of Low-Correlation Data Groups
6.3 Representing Data Correlation with Heatmaps
6.4 Visualization of High-Correlation Data
Chapter 7 Data Processing of Mold Temperature

7.1 Determination of Production Shifts
7.2 Exploring the Relationship Among the Three Temperature Measurements
7.2.1 Relationship of the Three Temperature Measurements in the Overall Data
7.2.2 Relationship of the Three Temperature Measurements for Mold No. 8 Data
Chapter 8 Analysis of Production Data for Mold No. 8
8.1 Division of Data by Shift
8.2 Data Status of Each Shift
8.3 Relationship Between Mold Temperature and Cycle Time
8.3.1 Examining the Correlation Between Mold Temperature and Cycle Time with a Heatmap
8.3.2 Line Chart Visualization of the Correlation Between Mold Temperature and Cycle Time
Conclusion
Acknowledgments
References
Appendix A Raw Data
Appendix B Complete Program Code

List of Figures and Tables

List of Figures

Figure 1 Purpose of this thesis
Figure 2 Main components of a smart factory
Figure 3 Configuration of the Python development environment
Figure 4 Data displayed in the terminal
Figure 5 Output of Code Segment 4
Figure 6 Visualization of low-correlation data groups
Figure 7 Heatmap
Figure 8 Visualization of weakly correlated data
Figure 9 Existing data.
Figure 10 Visualization of operator IDs (OPER)
Figure 11 Visualization of mold codes (MOLD)
Figure 12 Shift analysis of the foundry
Figure 13 Relationship among the three temperature measurements for the full dataset
Figure 14 Data status of mold temperatures 1, 2, and 3
Figure 15 Relationship among the three temperature measurements for Mold No. 8
Figure 16 Heatmap of mold temperature and cycle time
Figure 17 Relationship between temperature and casting cycle time for Shift 5

List of Tables

Table 1 Industrial initiatives of countries other than the United States and Germany
Table 2 Current status of networked manufacturing and informatization in the casting industry
Table 3 Python extension libraries for data processing
Table 4 Mainstream deep learning frameworks supporting Python
Table 5 Data types of process parameters

Table 6 Shifts at the foundry
Table 7 Data status of mold temperatures 1, 2, and 3
Table 8 Basic data summary for Shift 1
Table 9 Basic data summary for Shift 2
Table 10 Basic data summary for Shift 3
Table 11 Basic data summary for Shift 4.
Table 12 Basic data summary for Shift 5

List of Code Segments

Code Segment 1 Importing libraries
Code Segment 2 Reading the data... 16
Code Segment 3 Renaming column indexes... 17
Code Segment 4 Counting data outliers... 18
Code Segment 5 Removing data outliers... 18
Code Segment 6 Visualizing low-correlation data groups... 19
Code Segment 7 Removing low-correlation data groups... 20
Code Segment 8 Plotting the heatmap... 21
Code Segment 9 Visualizing weakly correlated data... 22
Code Segment 10 Removing the holding furnace number data and the shift number (SHIFT) column... 23
Code Segment 11 Visualizing operator IDs (OPER)... 24
Code Segment 12 Visualizing mold codes (MOLD)... 24
Code Segment 13 Shift analysis of the foundry... 26
Code Segment 14 Relationship among the three temperature measurements for the full dataset... 27
Code Segment 15 Fluctuation in mold temperatures 1, 2, and 3... 28
Code Segment 16 Relationship among the three temperature measurements for Mold No. 8... 30
Code Segment 17 Dividing the data by shift... 32
Code Segment 18 Retrieving basic data information... 32
Code Segment 19 Plotting the heatmap for Shift 5... 34
Code Segment 20 Relationship between temperature and casting cycle time for Shift 5... 35

Design of Python-Based Casting Data Visualization

Major/Class: Metal Forming 1603 Student Name: Yang Ye

Supervisor: Zhao Guoping Title: Associate Professor

Abstract: In the context of Industry 4.0, the global industrial manufacturing sector is moving toward intelligent, digital, and integrated transformation. However, China's progress in the Fourth Industrial Revolution has been relatively slow, particularly in the casting industry—a traditional manufacturing sector characterized by complex processes, demanding production conditions, difficulties in information collection, and numerous interfering factors. To optimize casting quality and improve production efficiency, advancing the digitization and informatization of the foundry industry has become an urgent task. At present, the application of information technology in casting production remains limited in China. Python, as a programming language well suited to industrial applications, features concise syntax, ease of learning, and powerful capabilities, offering distinct advantages in data processing and visualization. Although Python has already been applied to manufacturing data platform construction, production data management, and embedded Internet of Things (IoT) systems, its application in casting data processing remains largely unexplored. The use of Python for process data analysis in the foundry industry therefore holds broad prospects and significant research value. This paper employs Python together with its extension libraries—Pandas, NumPy, and Matplotlib—to analyze and explore production data related to the casting process of aluminum cylinder heads in a foundry, with the findings presented in the form of visual charts. The results indicate that the basic production conditions of the foundry are stable and that the production of aluminum cylinder head castings generally meets process requirements.

Keywords: Foundry Python; Manufacturing; Industry 4.0; Data Visualization

Introduction

Against the backdrop of Industry 4.0, China's manufacturing sector is inevitably moving toward industrial intelligence, with information and data playing a pivotal role in this process. As a traditional manufacturing sector, the foundry industry has seen slow progress in its industrial intelligence transformation, particularly facing significant shortcomings in the collection and processing of process data.

Python, possesses unparalleled advantages in the fields of big data and artificial intelligence. It also offers unique strengths in data processing and visualization. While there are already precedents for applying Python to the construction of manufacturing data platforms, production data management, and IoT integration, its application to foundry process data processing remains unexplored. Applying Python to the foundry industry holds broad prospects and significant research value.

This paper will utilize Python, leveraging the data processing and visualization capabilities of the Pandas, NumPy, and Matplotlib libraries within a programming editor, to process, analyze, and visualize mold temperature data from aluminum cylinder heads produced at a foundry, thereby assessing the process stability of the foundry.

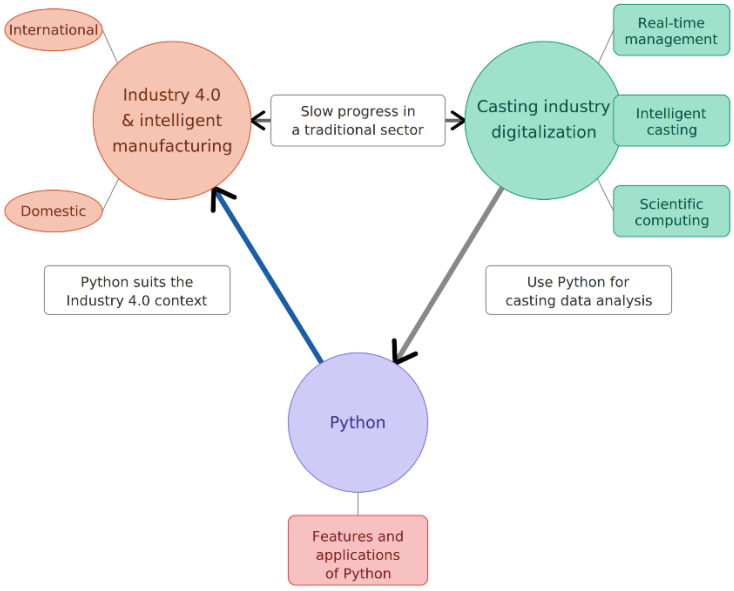


Figure1 Purpose of the Thesis

Chapter 1: Industry 4.0 and Smart Manufacturing

Following the "Electronic Age," where automation was achieved through informatization, we are now on the cusp of the Fourth Industrial Revolution—an era of transformative change driven by the application of data flow in manufacturing.Against the backdrop of Industry 4.0, manufacturing processes will rely on automation, robotics, and digital support for machine-to-machine communication and product management flows. Our target is to create a smart factory capable of end-to-end product production processes . In such a factory, everything from technical development and production planning to standardization and control will be integrated through value streams, enabling a significant shift from mass production to personalized production.

1.1 The Theory of Industry 4.0

Industry 4.0 encompasses concepts such as the Internet of Things (IoT), Cyber-Physical Production Systems (CPPS), and the Internet of Services (IoS).In the era of Industry 4.0, machines communicate in a fully automated manner. Within the context of smart factories, the physical and virtual worlds can be effectively integrated. Cyber-physical production systems monitor physical production processes and process the resulting digital data into a language that lower-level decision-making systems can understand. These cyber-physical systems can automatically observe production processes and respond in real time with great flexibility, thereby achieving a high degree of automation.The networked production environment is built upon the physical production environment. By installing intelligent sensing devices—such as sensors—on various objects and within environments throughout the production and processing workflow (e.g., production facilities, power systems, logistics systems, and products), and connecting these objects via networks to enable communication and interaction, a cyber-physical production system is formed. Through cloud computing technology, this cyber-physical production system is integrated with the Internet of Services, thereby establishing the smart factory:[5] .

In the smart manufacturing process, there exists a vast amount of diverse process data. Data circulation and network interconnectivity enable the high-speed flow of data and information, and big data generates immense value and impact from knowledge and data. Under these circumstances, there is a significant demand for data mining technology, Data mining technology which can be used for production outcome prediction, product defect analysis, process parameter optimization, and machine failure prediction.The vast majority of advanced modeling methods in data mining are used to design and control production processes[27] .

In smart factories , the capabilities of machines will be greatly enhanced; while communicating with each other, machines can perceive changes in related equipment and the working environment through other components, and machines can also exchange information with operators[5] . Machines capable of collaborative work and social communication among multiple objects become nodes in cyber-physical production systems.

In addition, the concept of “Operator 4.0”—that is, the augmented operator—has emerged in the context of Industry 4.0 [57].In manufacturing processes that are difficult to fully automate and require real-time analysis and interpretation of large amounts of data, the concept of “Operator 4.0” assumes that operators use various assistive technologies to support both physical and mental work. In this way, machines extend both human physical and mental capabilities. Relevant technologies include virtual reality and augmented reality (used for on-the-job training on the production line, as well as for displaying information about machine, production, and resource status, or decision-making prompts, directly at the production site).The key components and related concepts of a smart factory are shown in Figure 2.

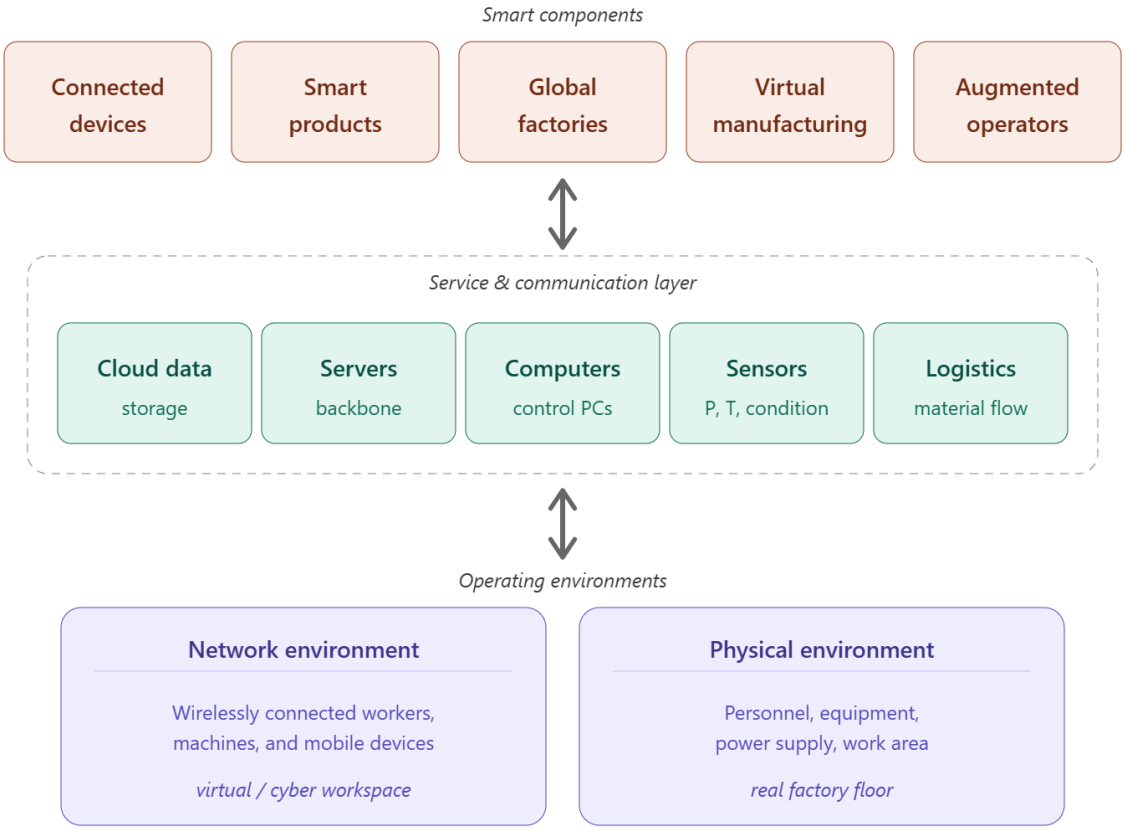


Figure2 Smart Factory Key Components

1.2 Digital Transformation of Manufacturing Abroad

Developed countries around the world have put forward their own visions and plans for the critical task of digital transformation in manufacturing. The most representative examples are Germany, a traditional manufacturing powerhouse, and the United States, the world leader in information technology. Given their respective unique strengths, the United States and Germany have adopted different priorities and reform approaches, each pursuing their own development paths.

Germany is undertaking a bottom-up structural reform that transitions from the physical to the digital realm. The country has positioned cyber-physical systems as the focal point and core of its industrial structural reform. First, it converts production factors such as industrial equipment, industrial components, raw materials, and personnel into digital form. These are then connected via the internet to form a digital chain, thereby integrating all necessary functions into a flat, integrated functional platform, which in

turn forms the Industrial Internet of Things (IIoT).The focus of U.S. manufacturing digitization lies in network interconnectivity, using network platforms as the foundation for industrial structural innovation. The United States positions the Industrial Internet as the focal point and core of industrial structural reform, primarily manifested through Industrial Internet platforms [1].The foundation of the U.S. manufacturing ecosystem lies in the Industrial Internet of Things (IIoT) platform, which enables data transmission and the rational allocation of resources through enterprise interfaces capable of accessing massive amounts of data on the internet platform. In this way, the basic network connecting industrial equipment, raw materials, operators, and information systems is integrated to form the IIoT platform.

During the Third Industrial Revolution, the United States and Germany, due to their respective national characteristics, had diverged in their information systems, with the former emphasizing the production floor and the latter emphasizing production management. However, in the Fourth Industrial Revolution, driven by new production demands and at a higher technological level, these two approaches have converged into a unified system. Correspondingly, other developed nations outside the U.S. and Germany have also proposed their own strategies and plans for industrial structural reform[10] .

Table1 Industrial plans of countries other than the US and Germany

Country	UK	France	Japan
Industry Plans	Intelligent Manufacturing Strategy and High-Value Manufacturing Plan	Future Smart Factory Initiative	Future New Industry and New Market Development Research Plan

1.3 The Era of Industry 4.0 Domestic Manufacturing

Overall, our intelligent transformation process is still in its early stages [8], and what we need to focus on right now is establishing process data collection and industrial system integration; Industry 4.0 is not yet a reality.

Compared to advanced countries internationally, the concept of smart manufacturing in China remains relatively underdeveloped. Although China’s existing information and communications technology (ICT) infrastructure is already quite well-established, the integration of digital information technology with industrial manufacturing remains severely inadequate [2]. The lack of deep integration between network information technology and manufacturing industrial applications is primarily reflected in the absence of a foundational process database, a shortage of specialized software, and the low level of digitization in manufacturing processes.The processing and analysis of production and process data also face numerous challenges in actual production processes. If the quality of collected data is poor, the information derived from analysis may fail to meet requirements. This can lead to distorted analysis results, rendering the entire process data processing and analysis process ineffective and making it difficult to achieve the intended outcomes [9]. The collection of process data holds significant research value and should be given due attention. If data collection capabilities cannot be improved, this will place higher demands on the level of data mining and modeling.

"Made in China 2025" explicitly identifies the advancement of intelligent and informatized manufacturing as the overarching direction for China’s industrial upgrading [7].To drive industrial innovation and development through smart manufacturing, we should draw on the industrial philosophies of advanced nations, leverage cutting-edge computer technology and existing capabilities to establish industrial innovation hubs, and ultimately achieve enterprise management spanning the entire product lifecycle, enabling decision-making across the full industrial production process[4] .

1.4 The Foundry Industry in the Context of and Smart Manufacturing

Due to the complex and challenging conditions involving machinery, personnel, and measurement equipment in the foundry industry, various activities performed or controlled manually may significantly disrupt the smooth implementation of IoT solutions. Therefore, process control at the foundry site is of utmost importance, and the informatization of quality control at the foundry site requires continuous improvement and optimization.

Data collection in the foundry industry is particularly challenging due to the inherent complexity and difficulty of the processes, the wide variety of products, and the high level of technical expertise required. The complexity of foundry processes is closely related to the number and types of target castings, as well as the number and types of casting defects. Issues regarding the coordination between different process stages and their mathematical description significantly limit the implementation of cyber-physical production system solutions.At present, it is virtually impossible to achieve a fully automated foundry that fully aligns with the concept of a smart factory; however, many production processes have recently been automated to some extent by reducing operator involvement. In foundry processes, despite the emergence of an increasing number of methods for auxiliary decision-making, human labor remains an indispensable part of the process. The data collection phase is critical because only by ensuring data integrity and understanding the collection frequency can mathematical models be established based on effective information.Careful consideration must be given to the processing of on-site information during the casting production process and subsequent data analysis to achieve the ultimate goal of networked collaborative manufacturing[24] .

Table2 Current Status of Networked Manufacturing and Informatization in the Foundry Industry

Category	Description
Current Situation – Advantages	Overall, the industry has entered a development phase and has shown rapid growth in recent years. A small number of foundry enterprises have implemented comprehensive process management for production quality, achieving full lifecycle traceability at the individual component level.
Current Situation – Disadvantages	Foundry enterprises generally have weak informatization capabilities. Integration and collaborative control of upstream and downstream industrial information are limited. The adoption rate of digital quality management and ERP (*Enterprise Resource Planning*) systems still needs improvement.

Chapter 2: Application of Information Technology in the Foundry Industry

Implementing Industry 4.0 processing solutions for our castings under current conditions will pose a unique challenge. Against the backdrop of the widespread adoption of smart manufacturing concepts, most foundries still lack the capability to process many types of process data. Foundry process information has become the most fundamental component of traceability data for castings at the time of delivery. The collection and processing of foundry process information should be given due attention ([10]).Existing information technologies are primarily applied to the foundry industry in the areas of real-time production management, intelligent casting systems, and data modeling.

2.1 Real-Time Production Management

In terms of building information platforms and real-time production management

systems, industry professionals have proposed various architectures and developed multiple systems, thereby improving data collection and production line management at foundry sites. Zhang Shaofang et al. established a centralized monitoring system based on multi-level network control, achieving visualized management of production across different foundry departments through the application of fieldbus technology . This has enhanced data collection and on-site management for discrete casting processes([16]).Zhang Fuqun et al., based on a wireless sensor network framework, implemented dynamic management of foundry production, ultimately strengthening monitoring of key equipment and improving collaboration among different foundry departments[17] . Liu Jinhai proposed new technical indicators for evaluating molten iron metallurgical quality, developed a metallurgical quality evaluation system based on thermal analysis technology , and created an intelligent control system for molten iron metallurgy[59] , which helps control casting quality by focusing on molten iron quality. Cheng Jingping analyzed a computer-integrated quality control system for DISA molding production lines, which enables dynamic quality control of real-time casting process information[23] . Zhu Xianbin et al. provided a relatively comprehensive collaborative platform framework design based on an actual mechanical manufacturing plant. They established a comprehensive production information management system[25] .Ghosh et al. investigated the application of DMAIC (Six Sigma Improvement) in online production using online process data under conditions where process parameters remained largely constant. They proposed a data mining-driven DMAIC framework for defect diagnosis and quality improvement. This framework effectively utilizes historical data and online process data during the casting process[28] .

2.2 Intelligent Casting System

Intelligent casting systems can self-improve and self-learn by comparing and analyzing production environment information with process-specific data, serving as an industry knowledge base powered by big data technology [20]. The concept of intelligent casting encompasses the definition of virtual casting, which primarily consists of:

tooling, selection of casting methods and mold parting lines, process flowchart design, and core design;Methodology: determining the casting’s gating system and designing its type and positioning; Casting: in this phase, the system simulates mold filling and solidification, predicts ICT-related casting defects, and performs cost calculations for furnace charge mixtures; Planning: the casting process design module, which involves determining process parameters, mold manufacturing, core making, melting, casting formation and finishing operations, material requirements planning, and casting cost estimation.

Based on the various process parameters required in the casting production process, Wang Zhiping et al. organically integrated dynamic and static data. Through data analysis and computation[18] , they constructed a casting process database to optimize processes and guide production. Du Yuzhu et al. analyzed and described the architecture of an intelligent casting information platform, outlining the concepts of casting product lifecycle management and MES (Manufacturing Execution System) for casting manufacturing enterprises [19].They analyzed the various modules within the system, which enables production and quality process traceability, thereby enhancing the enterprise’s production efficiency. Liu Xiangde, Jing Tao, and others studied the integration of computer simulation technology with network technology, attempting to apply the dynamic server-side scripting model to the foundry industry[22] . They created a network computing service platform for the foundry industry.Ravi, B., Datta, and others proposed a national collaborative initiative to create a web-based integrated foundry platform—a virtual foundry accessible via the internet. They identified two key technologies required for a virtual foundry system: automation and foundry simulation, as well as web-based project management throughout the entire foundry process [29].This enables virtual foundry trials to optimize tools, methods, and process parameters, allowing even small foundries in remote areas to easily utilize and benefit from the system.

2.3 Casting Data Modeling

From the perspective of data modeling, researchers have effectively integrated information technology with actual production. Rojek, G. applied artificial intelligence methods to casting quality assurance, utilizing tools such as artificial neural networks and case-based reasoning[30] .Jacek Kozłowski et al. conducted modeling and analysis of die-casting process parameters, concluding that the parameter with the greatest impact on die-cast part quality is the water temperature in the two cooling circuits. This research helps reduce the proportion of defective castings in the die-casting process[26] .Perzyk et al. proposed a method based on the concept of statistical process control, using advanced input-output regression models for process simulation to identify the causes of out-of-control signals in process output. This approach simplifies the identification of problematic processes and improves casting quality[31] . Subsequently, he utilized artificial neural networks to analyze the factors leading to porosity defects in cast steel, taking into account a large number of randomly varying production parameters such as process parameters, materials used, and the workers involved.In wet sand molding, Perzyk utilized Special Cause Control Charts (SCC) to analyze casting data, taking into account data autocorrelation. He concluded that SCCs are superior to traditional Statistical Process Control (SPC) in effectively reducing the number of signals and minimizing false signals caused by predictable factors. This research was applied during the casting data cleaning stage[33] .

Chapter 3: Features of Python

The Python programming language is considered one of the computer programming languages closest to natural language[44] . Its programming platform is free, easy to install, and undergoes constant updates and expansion. Additionally, compared to other languages, its syntax is very lightweight, allowing users to conveniently accomplish functions that would require many complex statements in other

languages using only a few lines of code. It is the most intuitive language for solving computer problems[58] .

Furthermore, due to the language’s open-source nature, Python users have developed hundreds of thousands of extension tools and toolkits to simplify problem-solving processes. Many of Python’s features rely on these toolkits, and thanks to its vast third-party libraries, Python’s applications span virtually every field, particularly in big data processing.

Table:3 Python data processing libraries

Extensions	Introduction
NumPy	Provides support for numerical arrays and efficient data processing functions
SciPy	Provides matrix operations and modules for advanced numerical computations
Matplotlib	Comprehensive data visualization and plotting library
pandas	Primary tool for data analysis, processing, and exploration
scikit-learn	Used for machine learning; supports regression, clustering, model analysis, and classification

3.1 Wide-ranging Applications

Its primary applications can be categorized into several major areas: developing web applications and building platforms using the Django framework; performing data analysis and exploration through a wealth of extension libraries; and, in the field of artificial intelligence, using Python’s extension libraries to easily implement convolutional neural networks for image recognition[44] .Python is also commonly used in software development, typically for simulation software and the secondary development of CAD software. Python is highly embeddable [46]and, because it does not require compilation, it is portable. Known as a “glue language,” it is often used for mixed programming in addition to standalone use. Furthermore, critical components can be written in C/C++ to improve the runtime efficiency and enhance the functionality of Python programs.

3.2 Data Processing Applications

Python possesses unparalleled advantages in the fields of big data and artificial intelligence. It is highly capable, offering unique strengths throughout the entire process—from data collection and processing to the final stage of data visualization[50] .

3.21 Scientific Computing

The Python programming language is concise and efficient, focusing on solving computational problems. Because it is simple and easy to learn, it is suitable for solving problems in non-computer science disciplines. It is well-suited for engineers and researchers to analyze their experimental or engineering data[45] . Python is also frequently used for secondary development of simulation software for multiscale simulations and process simulation.

3.22 Enterprise Operations and Maintenance

First, Python can be used for data analysis and also has the capability to build products, which simplifies workflows[14] . Second, as an interpretive language, Python allows for quick program modifications without the need for debugging, which significantly improves the efficiency of daily data operations and data management tasks[23] .

3.23 Data Visualization

Python possesses unique advantages in the field of data visualization and is well-suited for handling the vast and diverse datasets encountered in industrial mass production. Visualizing large-scale, complex data during industrial production processes holds significant practical importance [12]. Data visualization is not an isolated concept; it corresponds to other functionalities in Python and is closely linked to many cutting-edge disciplines in information technology.

Data in industrial production is highly complex and diverse; generally, useful information cannot be intuitively derived from raw data, making data visualization essential. In industrial production, in addition to structured data, there is often a large volume of various types of unstructured data that requires processing. Python’s

capabilities align well with industrial needs[13] .Technical experts have utilized Python to efficiently analyze both structured and unstructured data, ultimately presenting both types in a clear and concise manner [52]. By constructing data analysis and visualization platforms using Python's Django framework, they have visually presented structured data through charts.

3.3 Applications in Artificial Intelligence

Python plays a significant role as the mainstream programming language in the fields of artificial intelligence and has a wide range of applications [53]. Python's AI libraries —including PyWren, Tfdeploy, Kubelib, and PyTorch—provide robust support for artificial intelligence . Machine learning is a branch of artificial intelligence that focuses on learning from data; it uses computational methods to improve systems through experience [54]. Currently, Python also holds a clear advantage in the field of deep learning. TensorFlow is the most popular open-source deep learning framework today, and numerous excellent open-source deep learning platforms and frameworks are equipped with Python interfaces. In addition, many mainstream frameworks support Python [51] .

Table4 Mainstream frameworks supporting Python Deep learning

Framework	Technical Features
TensorFlow	Google's second-generation open-source deep learning technology. It can be applied to Google Search, image recognition, and speech recognition tasks.
Caffe	Developed by the BAIR and BVLC community. A widely used deep learning framework and one of the first mainstream industrial deep learning tools.
Theano	Gave rise to frameworks like Blocks. Highly flexible and suitable for academic research. Supports recurrent networks and language modeling, but has relatively slower performance.
Lasagne	A lightweight neural network library based on Theano. Supports feedforward neural networks, simplifies operations, and allows relatively low-level control.
Keras	Provides one of the most user-friendly APIs. Supports convolutional and recurrent neural networks, as well as stacked or graph-structured models.
MXNet	An open-source, flexible, and portable deep learning framework. Currently recommended by AWS as an official deep learning framework.
Chainer	Uses a dynamic computation graph (define-by-run). Stores computation processes instead of fixed network structures, enabling flexible use of Python control flow.

Chapter 4: Python and Industry 4.0

The key drivers of Industry 4.0 development include mobile communication and digital access channels for customers, as well as sufficient datasets and tools for data analysis. Amid the major trend of transitioning from labor-intensive centralized production to decentralized, flexible production and advancing human-machine collaboration, programming will be widely adopted in future industries. As a language suitable for the Internet of Things (IoT) and conducive to data processing, the application of Python in intelligent manufacturing is an inevitable trend.Under the concept of Industry 4.0, it is necessary to analyze the role Python plays in the process of industrial intelligence and digitization, and to explore its practical applications.

4.1 Python and the Internet of Things

The integration of Python into the construction of smart factories primarily relies on using Python-based web frameworks to build the layout of information-driven factories, or utilizing MicroPython—an embedded system for the Internet of Things (IoT)—to design smart terminals[47] . Python is also used for industrial control information management.

4.11 Building an IoT Platform

Yang Wushuai et al. established an IoT device management system, with smartwatches serving as the smart terminals. They utilized Python to build this IoT device management system, which enables data exchange between devices and provides device management functions[49] .

4.12 MicroPython

In the development of IoT embedded systems , C is the mainstream development language for low-level control , but it is difficult to master. In contrast, Python, which has simpler syntax yet can fulfill functional requirements, is well-suited for device control. Embedded systems that support Python include Zerynth, PyMite, and MicroPython.

MicroPython is a Python compiler that runs on microcontroller hardware, enabling low-level control of the chip [47].Chen Guanghui applied Python to the low-level control of IoT devices; based on MicroPython, he developed a temperature and humidity data acquisition system for laptops by integrating sensors and a display.

4.13 Industrial Control Information Management with

Engineering technicians can use Python for real-time management of industrial control data. Chen Kaicong and others designed a solution for multi-machine communication issues between the lower-level devices (PLCs and inkjet printers) of a keel forming machine control system and the industrial control computer. Through socket programming and Python, they exchanged data, achieving bidirectional data exchange between the industrial control computer and the lower-level devices[55] .

4.2 Building a Data Analysis Platform

Industry professionals both domestically and internationally have effectively utilized Python to build platforms, demonstrating its significant role in the manufacturing sector. Establishing a data analysis platform facilitates networked collaborative manufacturing, enables innovation in enterprise collaboration methods, and transforms and upgrades existing traditional manufacturing production models·[11] .

Engineering and technical personnel, having analyzed the importance of data in the manufacturing and processing industries, as well as current typical data processing tools, have created a multifunctional platform[14] that provides information services for the processing industry. This platform enables rapid and efficient data mining and analysis of large volumes of manufacturing process data, presenting the results in clear graphical displays. Xu Zhong et al. utilized Python data analysis to construct a steel production system, integrating big data applications with industrial control to advance information technology development in the steel industry[56] .

4.3 Process Simulation

In addition to building platform systems directly applied to smart factory construction, Python is frequently used for process simulation during production and processing. This typically involves secondary development of simulation software using Python, encompassing multi-scale simulations of the entire processing workflow, contact stress simulations, molecular-level material structure simulations, and milling process simulations. By developing custom software packages, simulation efficiency is enhanced and the analysis process is simplified.

Chapter 5: Initial Setup for Data Processing

This paper was developed using an editor based on Python 3.76 , and employs Anaconda 3.7 , which integrates third-party libraries required for data analysis . It analyzes and explores production data related to the casting process of aluminum cylinder head castings at a foundry, presenting the results in the form of visual charts to outline the foundry's basic production status. This chapter serves as a preparatory phase, focusing primarily on configuring the Python development environment and providing a brief introduction to the raw data.

5.1 Initial Setup

Install Python 3.7.6 and Anaconda 3.7, and configure the development environment and working conditions for the editor.

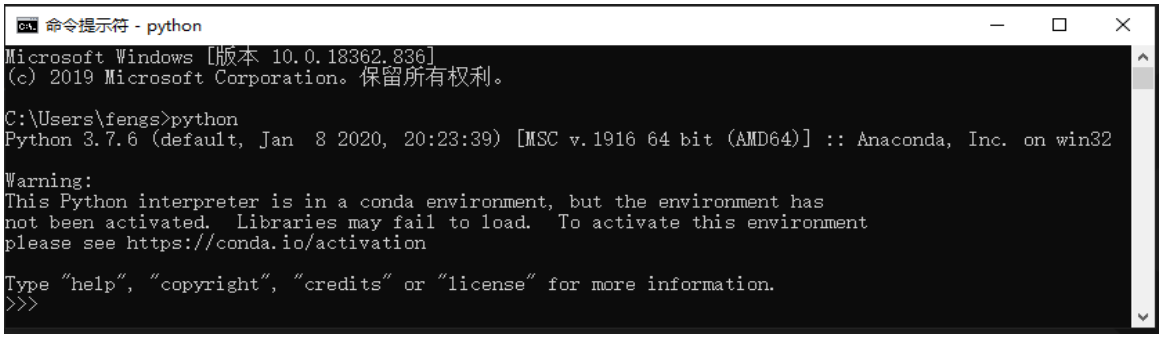


Figure3 Python Development Environment Configuration

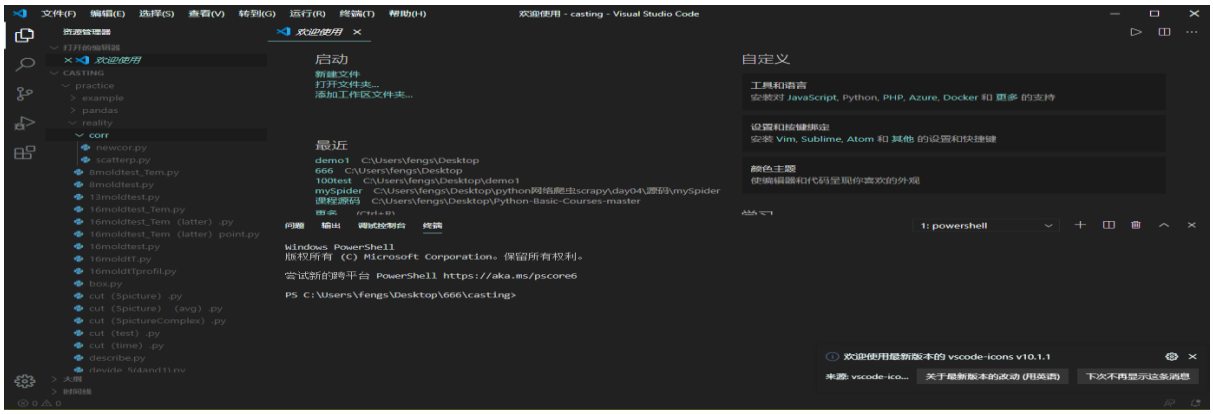


Figure 5: VSCode Work Interface

5.2 Basic Data Information

Initial review of foundry data. The raw data is provided in Appendix A. Each data set provided by the foundry contains 18 process parameters without units, including date and time, shift number, mold code, operator ID, serial number, holding furnace number, molten aluminum temperature, solidification time, mold temperature 1–3, and cooling water 1–7. As shown in the table below, there are a total of 902 sets of production data.

Table5 Process Parameter Data Types

Parameter Name	Parameter Category	Data Type	Numeric range
Date and time	Timestamp	object	04 17, 2020, 7:57 AM – 04 19, 2020, 6:33 PM
Shift Number	Code	float64	1, 2, 3
Mold code	Code	float64	3, 16, 18
Operator ID	Code	float64	1, 2, 3, 4
Serial Number	Code	float64	1 – 162
Holding Furnace No	Code	float64	1, 2, 3
Molten Al T	Temperature	float64	715 – 727
Curing Time	Duration	float64	240
Mold T 1-3	Temperature	float64	154 – 32767
Cooling water 1-7	Temperature	float64	139 – 141

Based on the actual casting production process, we preliminarily determine that each timestamp represents a measurement at a specific time point . A temperature measurement may occur after a single aluminum alloy pour, with the key data being the mold temperature. For mold temperature, each measurement yields three sets of temperature values from three measurement points on the same mold; we infer that the unit of mold temperature is ℃ .

5.3 Introduction and Import of the Extension Library

5.31 Brief Introduction to the Database

The main extension libraries to be imported are the following three.

Numpy: The Python language lacks native array processing capabilities, offering only "lists" as a similar alternative. Processing data using Python’s list functionality alone is time-consuming. When dealing with large datasets, Python’s basic list operations are insufficient for data processing. Numpy provides professional array functionality and is equipped with comprehensive built-in functions for efficient computation. Using Numpy enables effective storage and manipulation of numerical arrays.This library is the core of the Python data science ecosystem and serves as a critical dependency for other third-party libraries.

The Pandas library contains a wealth of built-in functions for efficient data processing, enabling rapid handling of datasets. It is also the primary tool used in this analysis of casting process data. With three fundamental data structures—series (array-like data sequences), DataFrames (combinations of multiple series), and indices (for retrieval)—Pandas’ arrays and matrices are labeled. Unlike NumPy, which relies on simple integer indices, Pandas significantly enriches data processing capabilities.

Matplotlib is the most commonly used data visualization tool in Python. Matplotlib This library also relies on the array foundation provided by the NumPy library and is a cross-platform data visualization library; its excellent cross-platform compatibility makes it extremely convenient to use.In subsequent data analysis, Matplotlib is used to visually

display relationships and trends among data, serving as the primary means of presenting analysis results. Common plot types include line charts, scatter plots, box plots, and heatmaps.

5.32 Importing Data and Databases

Since the original data file is in Microsoft Excel format, we use the `read_xlsx` function from the pandas library to import the data, converting the contents of the original file into a DataFrame format for subsequent array processing. Text following the `#` symbol is treated as a comment by the editor and is ignored.

See code snippets 1 and 2 below.

Code snippet1 : Importing the database

```
import pandas as pd#导入 pandas
import matplotlib.pyplot as plt#导入 matplotlib 绘图工具
import matplotlib
plt.rcParams['font.family'] = 'sans-serif'#绘图字体以及绘图风格设置
plt.rcParams['axes.unicode_minus'] = False#绘图显示负号
plt.rcParams['font.sans-serif'] = ['SimHei']#绘图中文标签正常显示
import numpy as np #导入 numpy
```

Code snippet2 : Importing the database Code snippet : Reading the data

```
df=pd.read_excel("C:/Users/fengs/Desktop/2020.xlsx", parse_dates=['日期与时间'])
#'日期与时间'序列以时间格式，从文件保存位置读入全部数据

print(df) # Print the imported data
```

Figure4 : Data displayed in the terminal

After executing code snippets 1 and 2 , the read data was output to the terminal. Since the total data was too long, the terminal displayed only the first and last five groups, for a total of ten data groups.

Code segment: Renaming the index of the `3 ` column

```
df.rename(
columns=
{
"Date and Time": "TIME",
"Mold Code": "MOLD", "Operator ID": "OPER",
 'Flow Number': 'FLOW', 'Shift Number': 'SHIFT'
"Mold Temp 1": "T1", "Mold Temp 2": "T2", "Mold Temp 3": "T3"}, inplace=True)# Rename column indices
```

Chapter 6: Data Correlation Analysis

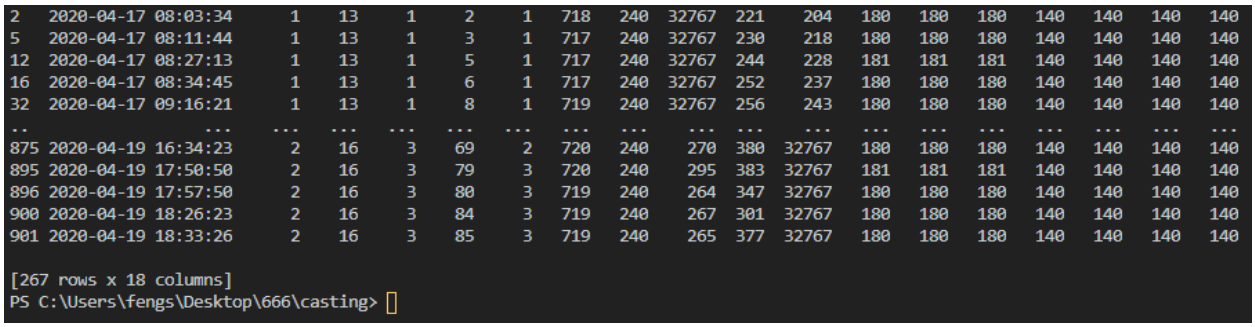
This chapter explores the production-related correlations between the following 8 types of data: (shift number, mold code, operator ID), (serial number,), (holding furnace number, molten aluminum temperature, curing time, cooling water 1-7), and (mold temperature measurements).

6.1 Clearing Data Overflow Values

First, after filtering out obvious data overflow values , we directly examined the mold temperature data for zones 1, 2, and 3. We found a large number of temperature readings at , 32767, and . These values are excessively high, severely impairing the visual representation of mold temperatures. Furthermore, since the molten aluminum temperature is maintained at approximately , 720, and℃ during the casting process, it is impossible for the mold temperature to exceed these values.A literature search revealed that 32767 is the maximum value that a programmable logic controller can represent for a 16-bit signed number. A displayed value of 32767 indicates a data overflow here, which should be cleared. Temperature values higher than 720℃ are also clearly data acquisition errors and should be cleared.

Program Segment4 Number of Data Overflows

```
print(df.loc[(df.T1 == 32767) | (df.T2 == 32767 ) | (df.T3 == 32767)])
# Check how many time points have data overflow at
```



```
2   2020-04-17 08:03:34     1    13     1     2     1   718   240  32767  221    204   180   180   180   140   140   140   140
5   2020-04-17 08:11:44     1    13     1     3     1   717   240  32767  230    218   180   180   180   140   140   140   140
12  2020-04-17 08:27:13     1    13     1     5     1   717   240  32767  244    228   181   181   181   140   140   140   140
16  2020-04-17 08:34:45     1    13     1     6     1   717   240  32767  252    237   180   180   180   140   140   140   140
32  2020-04-17 09:16:21     1    13     1     8     1   719   240  32767  256    243   180   180   180   140   140   140   140
..                  ...   ...   ...   ...   ...   ...   ...   ...    ...  ...    ...   ...   ...   ...   ...   ...   ...   ...
875 2020-04-19 16:34:23     2    16     3    69     2   720   240    270  380  32767   180   180   180   140   140   140   140
895 2020-04-19 17:50:50     2    16     3    79     3   720   240    295  383  32767   181   181   181   140   140   140   140
896 2020-04-19 17:57:50     2    16     3    80     3   719   240    264  347  32767   180   180   180   140   140   140   140
900 2020-04-19 18:26:23     2    16     3    84     3   719   240    267  301  32767   180   180   180   140   140   140   140
901 2020-04-19 18:33:26     2    16     3    85     3   719   240    265  377  32767   180   180   180   140   140   140   140

[267 rows x 18 columns]
PS C:\Users\fengs\Desktop\666\casting>
```

Figure 5: Results of running Program Segment 4 for `5 `

Executing code segments 3 and 4 reveals that among the 902 data sets, 267 contain a mold temperature measurement value of 32767, indicating a data overflow.

Program Segment5 : Clearing Data Overflow Values

```
df['T1'][df['T1'] >720] = None
df['T2'][df['T2'] > 720] = None
df['T3'][df['T3'] > 720] = None # Remove mold temperature data overflow values
```

After removing obvious outliers from the process data, assess its correlation with mold temperature measurements. Here, any value greater than 720 (℃) is considered an outlier.

6.2 Visualization of low-correlation data sets

It can be clearly observed that the data sets for molten aluminum temperature, solidification time, and cooling water (groups 1 – 7) exhibit very little fluctuation regardless of how mold temperature changes. It is highly likely that these variations are unrelated to changes in mold temperature . We will now verify this using visualization.

Program segment6 displays a plot of the low-correlation data set

```
plt.subplot(411)                                                    # Plot the molten
aluminum temperature for
plt.plot(df['TIME'], df['Molten Aluminum Temperature'],
```

```
  lw=0.8, color='r', label='Molten Aluminum Temperature')
  plt.ylabel('Temperature/°C')
  plt.title('Temperature')
  plt.grid(linestyle=':', color='r')
  plt.legend(loc='upperleft', bbox_to_anchor=(0.05, 0.95), ncol=4,
  title='Temperature Type',shadow=True,fancybox=True)
  plt.subplot(412)                                          # High-
Temperature Cooling Water Visualization
  plt.plot(df['TIME'], df['Cooling Water 1'],
  lw=0.8, color='indigo', label='Cooling Water 1')
  …….
  plt.subplot(413)                                          # Low-temperature
cooling water plot
  plt.plot(df['TIME'], df['Cooling Water 4'],
  lw=0.8, color='g', label='Cooling Water 4')
  ……
  plt.subplot(414)                                           # Curing time
plot
  plt.plot(df['TIME'], df['Curing Time'],
  lw=0.8, color='darkorange', label='Curing Time')
  plt.ylim(230, 250)
  plt.ylabel('Curing Time/sec')
  ……
  plt.show()
  Note: Similar or repeated sections of the program have been replaced with ellipses.
```

The following four figures, in order, are: (molten aluminum temperature), (high-temperature cooling water), (low-temperature cooling water), and (curing time). Observation reveals that the molten aluminum temperature fluctuates between 715° C and 725℃ ; the high-temperature cooling water (i.e., cooling water 1, 2, and 3) fluctuates between 180° C and 181° C ;Low-temperature cooling water (i.e., cooling water 4, 5, 6, and 7) fluctuates between 139–141°C, and the curing time remains constant at 240 seconds. Therefore, it can be concluded that these sets of data are unrelated to changes in mold temperature and do not require further analysis.



Figure6 shows a group of data with low correlation

These data sets are unrelated to changes in mold temperature and will not be considered in the overall data analysis.

Program segment7 Delete low-correlation data sets

```
df.drop(['Molten Aluminum Temperature', 'Cooling Water 1', 'Cooling Water 2', 'Cooling Water 3',
 'coolant_4', 'coolant_5', 'coolant_6', 'coolant_7', 'curing_time'], axis=1, inplace=True)
# Remove these data columns
```

6.3 Heatmap to visualize data correlations

Analyze the remaining data sets: shift number (SHIFT), mold code (MOLD), operator number (OPER), flow number (FLOW), holding furnace number and determine whether there is a correlation between these data sets and mold temperatures (T1, T2,

T3) . For convenience, import the seaborn library to plot a concise heatmap.

Code snippet8 Plot the heatmap

```
import seaborn as sns                                    #补充一个 seaborn
扩展库
def corr_heat(df):
    dfData = abs(df.corr())
    plt.subplots(figsize=(9, 9))                         #设置画面大小
    sns.heatmap(dfData, annot=True, vmax=1, square=True, cmap="Blues")
    plt.show()
                                                         #创建热力图函数
corr_heat(df)
```

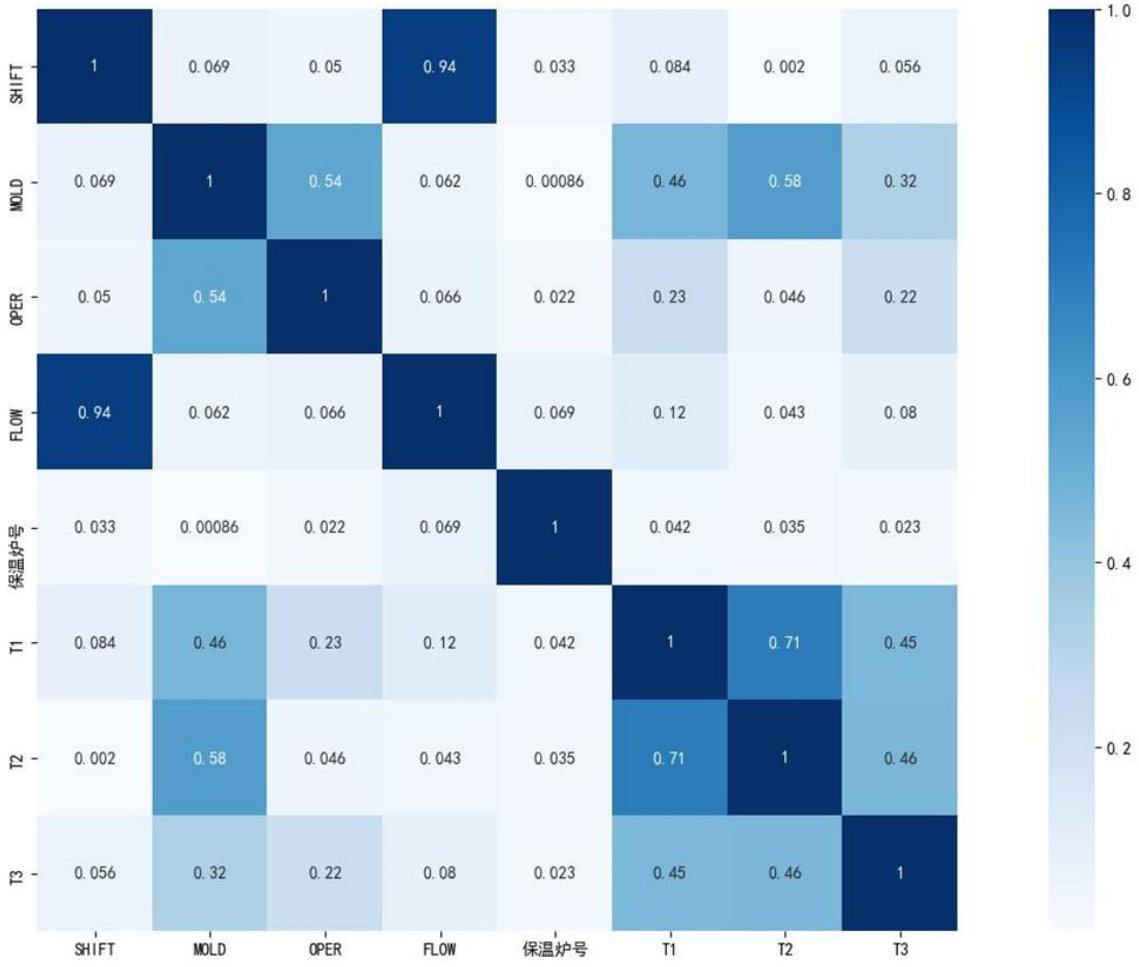


Figure7 Heatmap

As can be seen from the figure above, the following data sets show varying degrees of correlation with changes in mold temperature: (shift number), , (mold code), (operator number), (flow number), , (holding oven number), and .The correlation between the holding oven number dataset and the other datasets is very low, all below

0.1; shift number (SHIFT) shows a strong correlation with , flow number (FLOW) ; mold temperature (T1, T2, T3) has a very weak association with shift number (SHIFT) and can be disregarded ; Mold temperature (T1, T2, T3) has a certain correlation with flow number (FLOW) and should be taken into account; Mold temperature (T1, T2, T3) and Mold code (MOLD) and operator number (OPER) show a strong correlation; simultaneously, mold code (MOLD) and operator number (OPER) are strongly correlated with each other and must be taken into account.

Program segment9 Display weak correlation data plot

```
  plt.subplot(311) # Display the holding furnace number plot
  plt.plot(df['TIME'], df['Holding Furnace No.'],
           lw=0.8, color='crimson', label='Curing Time')
  plt.grid(linestyle=':', color='r')
  plt.ylabel('Holding Furnace No.')
  plt.title('Holding Furnace Number')
  plt.legend(loc='upper left', bbox_to_anchor=(0.05, 0.95), ncol=4,
             title='Holding Furnace Number', shadow = True, fancybox=True)
  plt.subplot(312) # Display the shift number plot
  ……
  plt.subplot(313) # Display serial number plot
……
  plt.show()
  Note: Sections of the program that are similar or repeated have been replaced with ellipses
```

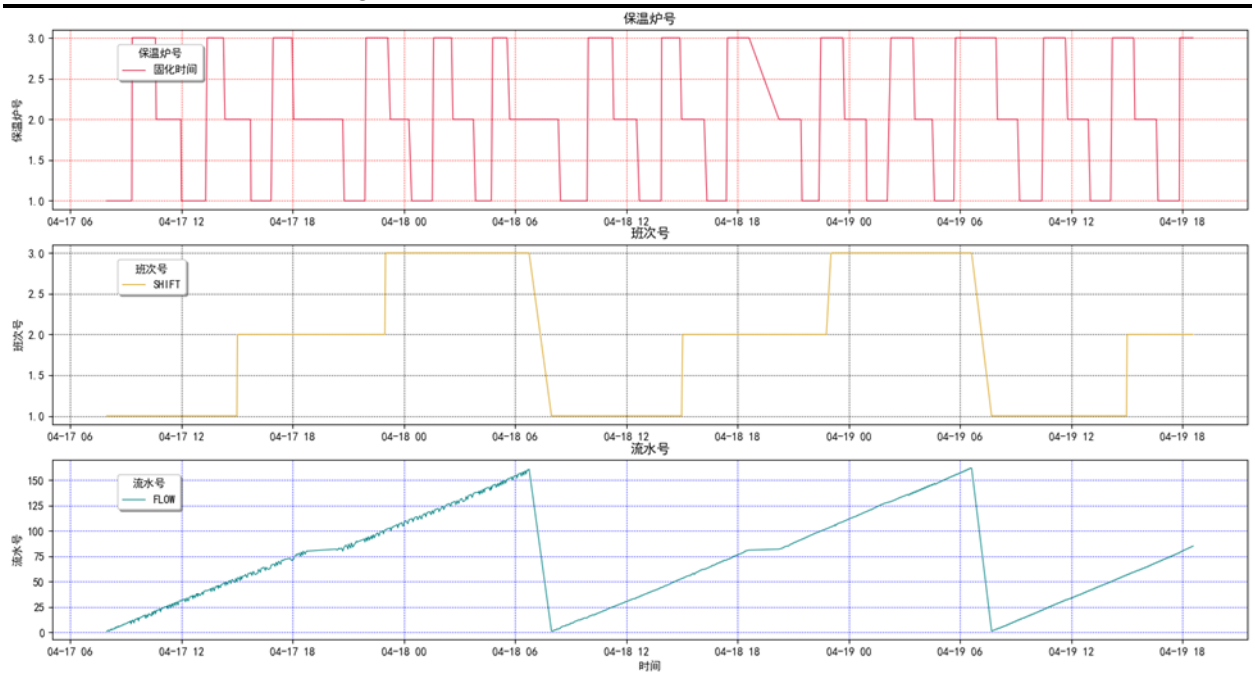


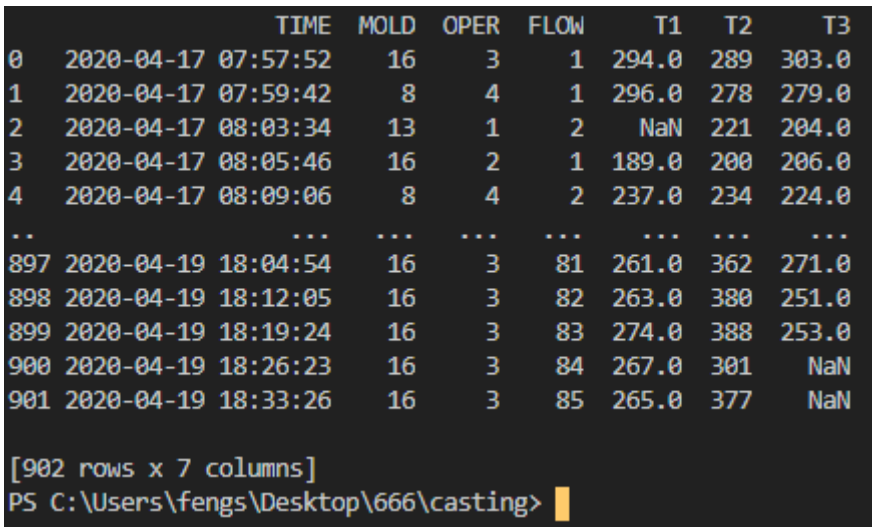
Figure8 : Visualization of Weakly Correlated Data

As shown in the figure above, the shift number (SHIFT) and holding furnace number simply vary periodically over time and have a weak correlation with mold temperature (T1, T2, T3); therefore, they will not be considered further.

Remove the data for the holding oven number and the columns for shift number (SHIFT) and . Continue analyzing the relationship between mold code (MOLD), operator number (OPER), and flow number (FLOW) and mold temperature (T1, T2, T3) .

Program segment10 Deletes the data for the holding oven number and the SHIFT column

```
df.drop(['Holding Oven ID', 'SHIFT'], axis=1, inplace=True)
# Remove the "Holding Furnace ID" and "SHIFT" columns from the data
```



	TIME	MOLD	OPER	FLOW	T1	T2	T3
0	2020-04-17 07:57:52	16	3	1	294.0	289	303.0
1	2020-04-17 07:59:42	8	4	1	296.0	278	279.0
2	2020-04-17 08:03:34	13	1	2	NaN	221	204.0
3	2020-04-17 08:05:46	16	2	1	189.0	200	206.0
4	2020-04-17 08:09:06	8	4	2	237.0	234	224.0
..	...	...	...	...	...	...	...
897	2020-04-19 18:04:54	16	3	81	261.0	362	271.0
898	2020-04-19 18:12:05	16	3	82	263.0	380	251.0
899	2020-04-19 18:19:24	16	3	83	274.0	388	253.0
900	2020-04-19 18:26:23	16	3	84	267.0	301	NaN
901	2020-04-19 18:33:26	16	3	85	265.0	377	NaN

[902 rows x 7 columns]
PS C:\Users\fengs\Desktop\666\casting>

Figure: Current data in `9 `

Currently, only the mold code (MOLD), operator ID (OPER), flow number (FLOW), mold temperature (T1, T2, T3), and date and time (TIME) are available. These seven sets of data require analysis.

6.4 Visualization of Highly Correlated Data

Mold code (MOLD) , Operator ID (OPER) , Flow number (FLOW), Mold temperature (T1, T2, T3) , and Date and time (TIME) data groups. In this section, we will categorize different operators and mold codes to observe their corresponding

relationships in the visualizations.

Program Segment11 Visualization of Operator ID (OPER)

```
plt.subplot(311)                                                    #模温 1
图显
plt.plot(df[df['OPER']==1].TIME,df[df['OPER']
==1].T1,lw=0.5,color='r',label='操作员 1')
……
plt.legend(loc='upperleft',bbox_to_anchor=(0.05,0.95),ncol=4,
title='OPER',shadow=True,fancybox=True)
plt.ylabel('温度/℃')
plt.title('模温 1')
plt.grid(linestyle=':',color='r')
plt.subplot(312)                                                    #模温 2
图显
……
plt.subplot(313)                                                    #模温 3
图显
……
plt.show()
```

Program Segment12 Mold Code (MOLD) Visualization

```
plt.subplot(311)                                                    #模温 1
图显
plt.plot(df[df['MOLD']==8].TIME,df[df['MOLD']
==8].T1,lw=1,color='g',label='模具 8')
……
plt.legend(loc='upperleft',bbox_to_anchor=(0.05,0.95),ncol=3,
title='MOLD',shadow=True,fancybox=True)
plt.ylabel('温度/℃')
plt.title('模温 1')
plt.grid(linestyle=':',color='r')
plt.subplot(312)                                                    #模温 2
图显
……
plt.subplot(313)                                                    #模温 3
图显
……
plt.show()
```

注：程序相似或者重复出现的部分用省略号替代

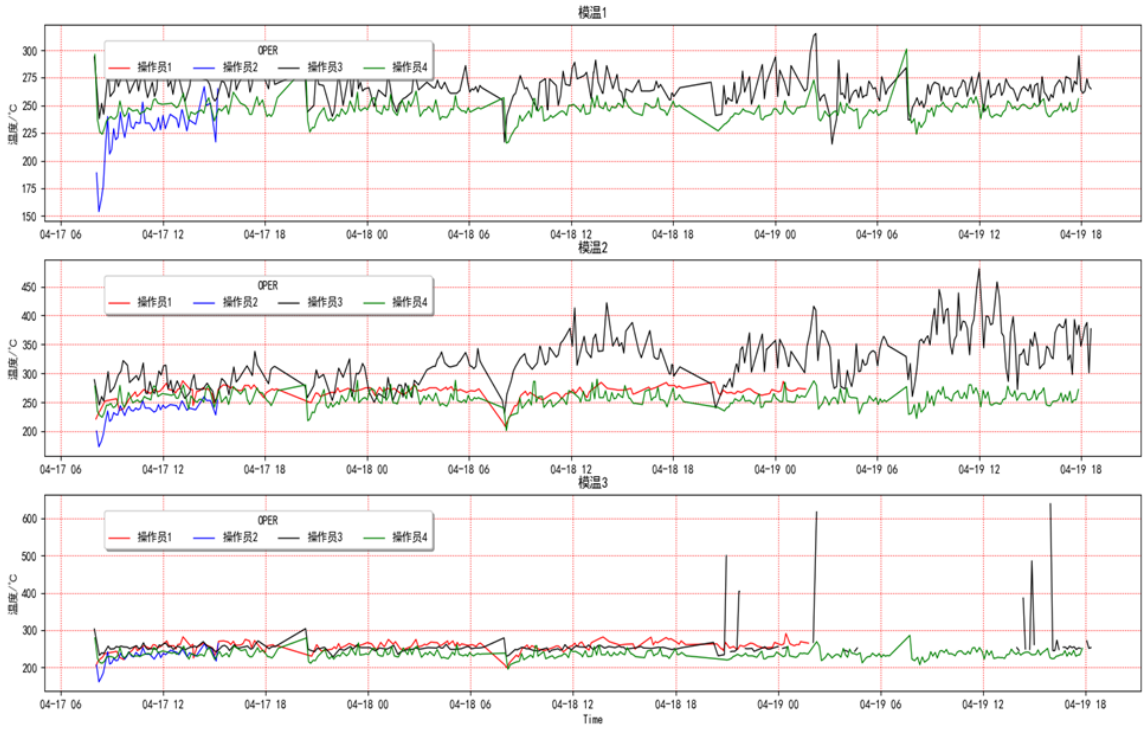


Figure10 Operator ID (OPER) display



Figure:11 Mold Code (MOLD) Display

A comparison of the two tables above reveals that, regarding mold temperature, the four operator codes correspond exactly to the three mold codes (: Mold 8 corresponds to Operator 4 ; Mold 13 corresponds to Operator 1; Molds 16 and 17 correspond to Operators 2 and 3, respectively).

Chapter 7: Data Processing for Mold Temperature

B ased on the conclusions from the previous chapter, it can be observed that the four operators are in perfect correspondence with the three mold codes. Among the overall mold temperature data (T1, T2, T3) at , the temperature data collected by operator 4 (corresponding to mold 8 at) at is the most complete and stable. In this chapter, based on an analysis of all data, we will further analyze and process the mold temperatures for mold 8, which has the best data quality.

7.1 Determination of Production Shifts

If each timestamp represents a temperature measurement taken after pouring into one of molds 8, 13, or 16, then production interruptions can be identified based on the maximum timestamp interval (i.e., the longest time span), and the duration of a continuous production shift can be determined. Add new columns : Time Increment Shift Interval (delta_T), Time Increment Shift Duration (gap).There are a total of 5 shifts; see Table 6 for detailed durations:

Program Segment13 : Foundry Shift Analysis

```
  delta_T = df['TIME'].diff()                                    # Sort by time
increment
  df['delta_T'] = delta_T
  print(delta_T.nlargest())
  print(df.sort_values(by='delta_T'))
    t = df.sort_values(by='delta_T')[897:902]
  l = t.sort_values(by='TIME')
  l['gap'] = l['TIME'].diff()
  print(l)
```

After running Code Segment 13, the foundry shift analysis is obtained, as shown in Figure 13:

	TIME	MOLD	OPER	FLOW	T1	T2	T3	delta_T	gap
0	2020-04-17 07:57:52	16	3	1	294.0	289	303.0	NaT	NaT
225	2020-04-17 20:20:11	16	3	82	281.0	279	304.0	01:36:32	12:22:19
399	2020-04-18 07:57:17	16	3	1	255.0	253	279.0	01:12:32	11:37:06
577	2020-04-18 20:13:26	16	3	82	271.0	282	267.0	01:36:46	12:16:09
739	2020-04-19 07:41:32	16	3	1	284.0	329	NaN	01:04:43	11:28:06

Figure12 : Foundry Shift Analysis

Table6 : Foundry Shift Schedule

Total Production	Temperature Measurements	Shift End Timestamp	Shift Interval	Shift Duration
225	225	April 17, 2020, 8:20:11 PM	01:36:32	12:22:19
399	174	April 18, 2020, 7:57:17 AM	01:12:32	11:37:06
577	178	April 18, 2020, 8:13:26 PM	01:36:46	12:16:09
739	162	April 19, 2020, 7:41:32 AM	01:04:43	11:28:06
902	163	April 19, 2020, 6:33:26 PM	----	09:47:09

Note: The first shift began at 2020-04-17 07:57:52, and the fifth shift ended when all data collection concluded; therefore, the interval for the fifth shift is not included

7.2 Exploring the Relationship Among the Three Temperature Measurements

Given that a single temperature measurement for the same mold yields three sets of mold temperatures (T1, T2, T3) , this section explores the relationship among these three temperature values using graphical representations for each shift as defined in the previous section. This section also introduces a new column: the average (avg) of the three sets of mold temperatures (T1, T2, T3).

7.21 Relationship between the three temperature measurements for the overall data

Code Segment:14 Relationship Among the Three Temperature Measurements for All Data

```
df['avg'] = df.total/3                    # Add the average temperature column
plt.subplot(511)                          # Display graph for Shift 1
```

```
  plt.plot(df['TIME'].loc[:224], df.T1.loc[:224], lw=0.8, color='r', label='T1')
  plt.plot(df['TIME'].loc[:224], df.T2.loc[:224], lw=0.8, color='g', label='T2')
  plt.plot(df['TIME'].loc[:224], df.T3.loc[:224], lw=0.8, color='b', label='T3')
  plt.plot(df['TIME'].loc[:224], df['avg'].loc[:224], lw=0.8, color='k', label='avg')
  plt.ylabel('Temperature (°C)')
  plt.title('Shift 1')
  plt.grid(linestyle=':', color='r')
plt.legend(loc='upperleft', bbox_to_anchor=(0.05, 0.95), ncol=4,
  title='Mold Temperature', shadow=True, fancybox=True)
  ……
  plt.subplot(515)                              # Display plots for runs 2 through 5
  ……
  plt.show()
```

Note: 1. If any of the mold temperatures (1, 2, 3) contain missing values, the average will also be a missing value. 2. Similar or repetitive sections of the code are replaced with ellipses.



Figure13 Relationship between the three temperature measurements for all data

Upon observing the figure above, it is found that in Shift 1 , Mold Temperatures 1, 2, and 3 exhibit the highest similarity, with most data falling within the 200–300℃ temperature range; Overall, the data for Mold Temperatures 1 and 2 are relatively consistent, with Mold Temperature 2 showing the greatest fluctuation and Mold Temperature 1 showing the least fluctuation. Below is a brief overview of the fluctuation

patterns for the three sets of data.

Code segment:15 Fluctuations in Mold Temperature 1, 2, and 3 data

```
  df['TG1'] = df['T1'].diff()                                    # Mold temperature
fluctuation
  df['TG2'] = df['T2'].diff()
  df['TG3'] = df['T3'].diff()
  print(df.TG1.describe(), df.TG2.describe(), df.TG3.describe())
  print(df.T1.describe(), df.T2.describe(), df.T3.describe())
```

```
count    555.000000
mean       0.935135
std       23.226777
min      -64.000000
25%      -17.000000
50%        0.000000
75%       18.500000
max       87.000000
Name: TG1, dtype: float64
count    901.000000
mean       0.097669
std       64.740056
min     -239.000000
25%      -34.000000
50%        0.000000
75%       36.000000
max      218.000000
Name: TG2, dtype: float64
count    731.000000
mean      -0.030096
std       48.588337
min     -416.000000
25%      -16.000000
50%        0.000000
75%       17.000000
max      413.000000
Name: TG3, dtype: float64
```

```
count    721.000000
mean     253.428571
std       16.489667
min      154.000000
25%      244.000000
50%      251.000000
75%      264.000000
max      315.000000
Name: T1, dtype: float64
count    902.000000
mean     281.288248
std       42.880898
min      173.000000
25%      254.000000
50%      269.000000
75%      299.000000
max      481.000000
Name: T2, dtype: float64
count    814.000000
mean     247.450860
std       33.046829
min      161.000000
25%      233.000000
50%      246.000000
75%      255.000000
max      639.000000
Name: T3, dtype: float64
```

Figure14 Mold temperature 1, 2, and 3 data

Table7 : Mold Temperature 1, 2, 3 Data

Mold Temperature Code	T1	T2	T3
Temperature Range/° C	145–315	173–418	161–639
Average temperature/° C	253	281	247
Maximum fluctuation range/° C	-64 – 87	-239 to 218	-416–413
Standard deviation of fluctuation/° C	23	64	48
Number of valid data points	721	902	814

From the table above, it can be seen that Mold Temp 1 , the degree of data fluctuation increases for data points 2 and 3, Mold Temp 2 all data is valid, and the most missing values are caused by overflow in Mold Temp 1 data.

7.21 Relationship Among the Measurement Values for Mold 8

The data set for Mold 8 operated by Operator No. 4 exhibits high data quality and good continuity, making it worthy of separate analysis. In this subsection, since the analysis focuses on data from a single mold, the mold run number is used as the x-axis, and the mold temperature data (and) is visualized by shift to observe process stability.

Section16 : Relationship between the three temperature measurements for Mold 8

```
df['FLOW']=np.arange(1,358)                                    #给模次编号
为流水
plt.subplot(511)                                               #班次 1
图显
plt.plot(df['FLOW'].loc[:65],df.T1.loc[:65],
lw=0.8,color='r',label='T1')
plt.plot(df['FLOW'].loc[:65],df.T2.loc[:65],
lw=0.8,color='g',label='T2')
plt.plot(df['FLOW'].loc[:65],df.T3.loc[:65],
lw=0.8,color='b',label='T3')
plt.plot(df['FLOW'].loc[:65],df['avg'].loc[:65],
lw=0.8,color='k',label='avg')
plt.ylabel('温度/C')
plt.title('班次 1')
plt.grid(linestyle=':',color='r')
plt.legend(loc='upperleft',bbox_to_anchor=(0.05,0.95),ncol=4,
title='模温',shadow=True,fancybox=True)
plt.subplot(512)                                               #班次 2
图显
……
plt.subplot(513)                                               #班次 3
图显
……
plt.subplot(514)                                               #班次 4
图显
……
plt.subplot(515)                                               #班次 5
图显
plt.plot(df['FLOW'].loc[279:358],df.T1.loc[279:358],
lw=0.8,color='r',label='T1')
……
plt.show()
注:1.一旦模温 1, 2, 3 当中存在空值,平均值即表现为空值 2.程序相似或者重复出现的部分
用省略号替代 3 模具 8 数据班次划分采取了与全部数据班次划分类似的方法,此处不多赘述
```

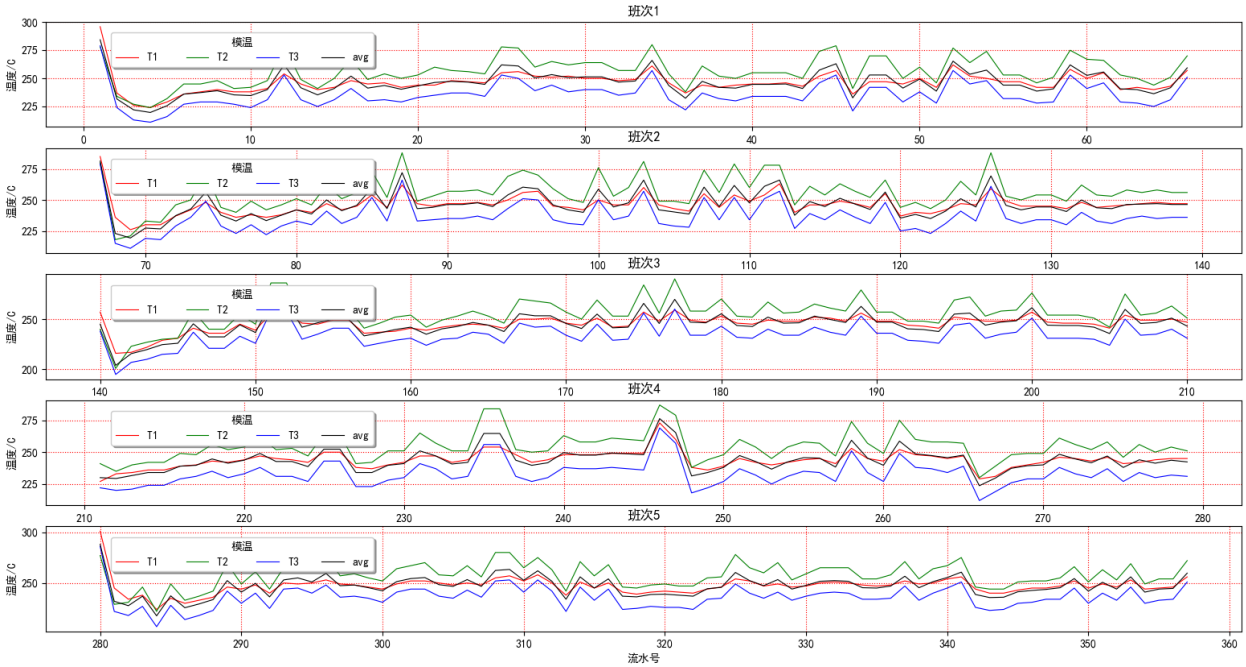


Figure15 : Relationship between the three temperature measurements for Mold 8

The data set for Mold 8 at is generally stable. At , Mold Temperature 1 , 2, and 3 show an approximately constant temperature difference. At , Mold Temperature 2 is greater than Mold Temperature 1, which is greater than Mold Temperature 3; the fluctuation patterns of all three are similar, and the overall average mold temperature is very close to that of Mold Temperature 1 at .

Chapter 8: Analysis of Production Data for Mold No. 8

For the data set regarding Mold 8 , the temperature differences between Mold 1, 2, and 3 appear to be approximately constant . The average of these three values closely resembles that of Mold 1, and the data is stable. This chapter continues the analysis of Mold 8’s mold temperatures and casting cycle times by shift.

8.1 Data Categorization by Shift

Since shift changes interrupt production, the mold temperature data and cycle

time data for Mold 8 are first divided into shifts before proceeding with shift-by-shift analysis.

Program segment17 for shift segmentation

```
shift1 = df[['delta_T', 'T1', 'T2', 'T3']].loc[1:65]                    #班次 1
shift2 = df[['delta_T', 'T1', 'T2', 'T3']].loc[66:138]                  #班次 2
shift3 = df[['delta_T', 'T1', 'T2', 'T3']].loc[139:209]                 #班次 3
shift4 = df[['delta_T', 'T1', 'T2', 'T3']].loc[210:278]                 #班次 4
shift5 = df[['delta_T', 'T1', 'T2', 'T3']].loc[279:358]          #班次 5
```

8.2 Data Status for Each Shift

The same method used to obtain basic data for the entire dataset was applied.

Program Segment18 : Retrieve Basic Data Information

```
print(shift1.describe())                                                #班次 1
print(shift2.describe())                                                #班次 2
print(shift3.describe())                                                #班次 3
print(shift4.describe())                                                #班次 4
print(shift5.describe())                                                #班次 5
```

Upon completion, the data was obtained and organized into the following 5 tables:

Table8 Basic Data for Shift 1

Parameters	**T1**	**T2**	**T3**	**Cycle Time(min:sec)**
Data Range/° C	224 – 265	224–280	211–257	1:40 – 3:26
Average/° C	245	255	235	09:54
RMS	65	--	--	--

Table9 : Basic Information on Shift 2 Data

Parameters	T1	T2	T3	Cycle Time (min:sec)
Data Range/° C	226 – 285	218 – 288	211 – 279	1:40 – 39:42
Average/° C	246	256	236	09:50
RMS	73	--	--	--

Table10 Basic Data for Shift 3

Parameters	T1	T2	T3	Cycle Time (min:sec)
DataRange/° C	216 - 259	201 - 290	195 - 260	1:21 - 8:23
Average/° C	245	255	233	09:54
RMS	71	--	--	--

Table11 : Basic Data for Shift 4

Parameters	T1	T2	T3	Cycle Time (min:sec)
Data Range/° C	227 - 273	230 - 287	212 - 269	1:30 - 9:28
Average/° C	243	253	244	10:21
RMS	69	--	--	--

Table12 Basic Data for Shift 5

Parameters	T1	T2	T3	Cycle Time (min:sec)
Data Range/° C	224 - 265	224 - 280	211 - 257	7:00 - 7:52
Average/° C	245	255	235	08:41
RMS	78	--	--	--

From the above data, it can be observed that based on the set of data from (Mold 8) , the average temperature difference between zones 1, 2, and 3 of Mold 8 remains within a fixed range of 10° C, indicating stable data; the average casting cycle time also remains between 8 and 10 minutes, indicating a very stable process.

8.3 Relationship Between Mold Temp and Cycle Time for Mold 8

Among the data for Mold 8 across all shifts, the data for Shift 5 has the highest number of valid values . Therefore, we will analyze the relationship between mold temperature and cycle time using the data from Shift 5.

8.31 Examining the Correlation Between Mold Temperature and Cycle Time Using a Heat Map

Create a concise heatmap.

Code snippet19 Plots a heatmap for Shift 5

```
shift5['usetime'] = shift5.delta_T.dt.total_seconds()
#将耗时单位全部换成秒
def corr_heat(df):
    dfData = abs(df.corr())
    plt.subplots(figsize=(9, 9))# 设置画面大小
    sns.heatmap(dfData, annot=True, vmax=1, square=True,
cmap="Blues")
    plt.show()#创建热力图函数

corr_heat(shift5)
```

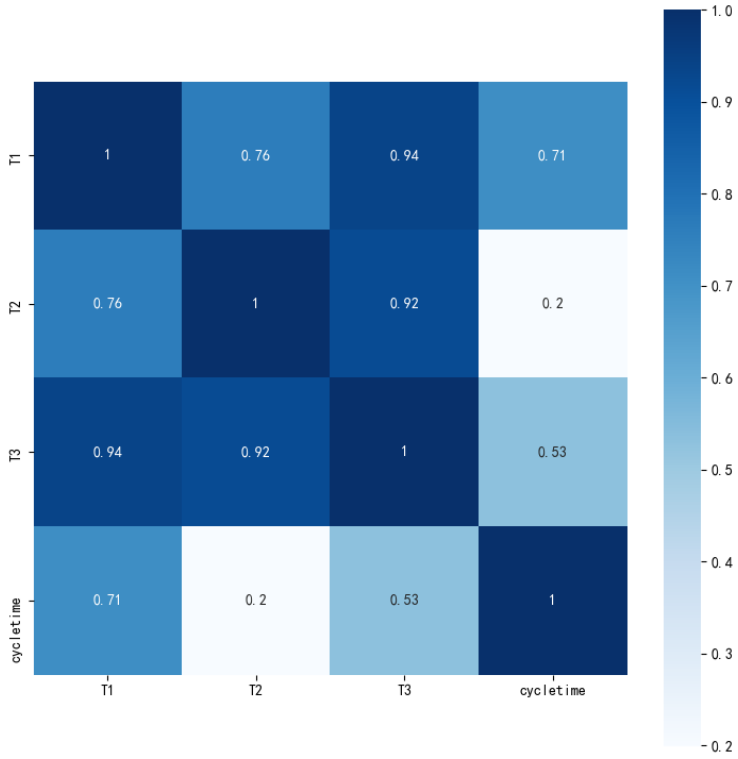


Figure16 Mold Temperature vs. Cycle Time Heatmap

This figure indicates that mold temperature is positively correlated with casting cycle time ; the longer the cycle time, the higher the mold temperature.

8.32 Visualization of the Line Chart Showing the Correlation Between Mold Temperature and Cycle Time

Compared to a line chart, a scatter plot is less susceptible to the influence of individual data points. Here, a scatter plot is combined with a line chart to observe the correlation between mold temperature and casting cycle time for Mold 8, Shift 5, , indicating that mold temperature is correlated with casting cycle time.

Code snippet20 Relationship between temperature and casting cycle time for Shift 5

```
  plt.plot(np.arange(78), shift5['cycletime'],
  lw=2,marker='o',color='k',label='Cycle/sec')                    #Cycle
  plt.plot(np.arange(78), shift5['T1'],
  lw=2,color='purple',label='Mold Temp 1/°C')                     #
Mold Temp
  plt.scatter(np.arange(78), shift5['T2'],
  marker='<', color='crimson', label='Mold Temp 2/°C')
  plt.scatter(np.arange(78), shift5['T3'],
  marker='<', color='steelblue', label='Mold Temp 3/°C')
  plt.grid(linestyle=':', color='r')
  plt.title('Mold Temperature vs. Cycle Time')
  plt.legend(loc='upperleft', bbox_to_anchor=(0.05, 0.95))
  plt.show()
```

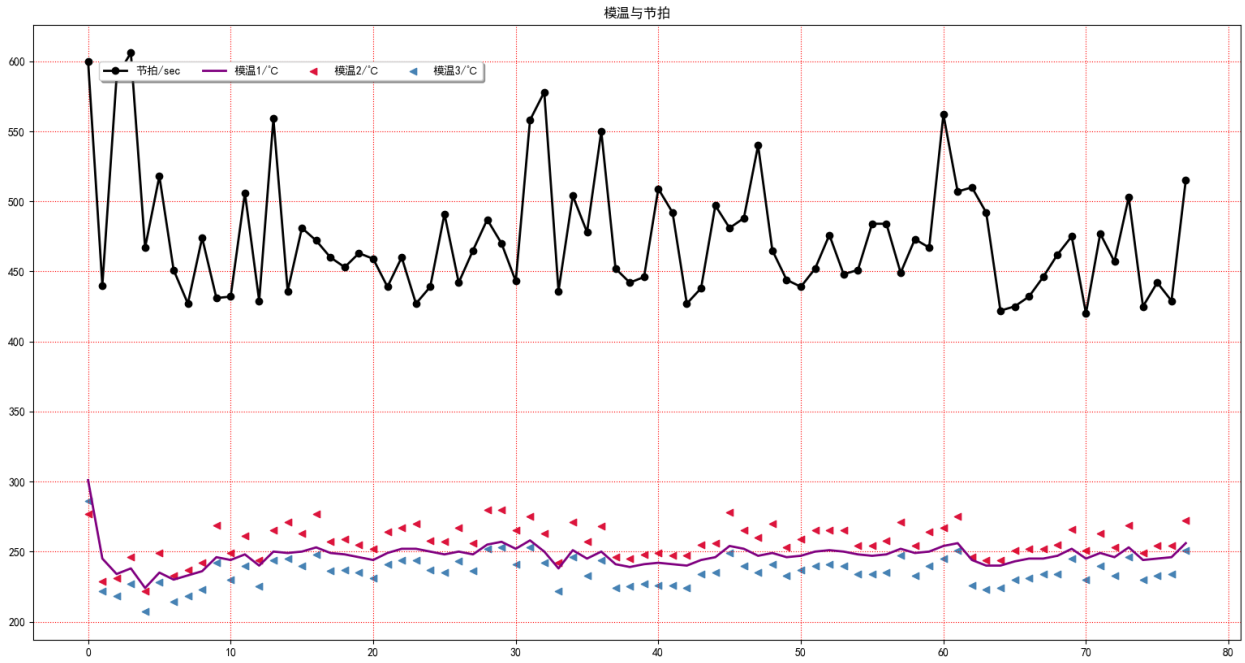


Figure17 : Relationship between temperature and casting cycle time for Shift 5

Conclusion

This paper first examines the transformation of manufacturing in the context of *Industry 4.0*, highlighting the growing demand for digitalization in the foundry industry and the suitability of Python for supporting Industry 4.0 applications. It argues that Python is poised to play a significant role in the digital transformation of the foundry sector and therefore holds substantial research value. However, there are currently very few precedents for applying Python to the analysis of production data in foundry environments.

Subsequently, this study employs the Python programming language, together with several efficient third-party libraries and a code editor, to analyze a dataset of mold temperature measurements collected from an aluminum cylinder head casting process. By leveraging array-processing and data visualization tools, an initial assessment of the foundry's production status was obtained. The results indicate that the production process is generally stable and standardized; however, the data acquisition system still requires further improvement.

Due to time constraints and limited expertise in advanced computing techniques, several limitations remain in the data analysis:

- **First**, the criteria used to identify data outliers were not sufficiently rigorous, and some anomalous data points were not thoroughly investigated.
- **Second**, the relationship between casting cycle time and mold temperature variation was not comprehensively analyzed.
- **Third**, contextual information was insufficient; limited understanding of on-site process conditions and production standards led to weak integration between data analysis and actual production practices.

Mold temperature plays a critical role in determining casting quality. Future work should focus on a deeper investigation of the casting process by comparing measured mold temperature data with simulation results (e.g., from ProCAST). Such integration would support process optimization and contribute to improved production quality.

Acknowledgments

During this unusual period, I was able to complete my graduation project after several months of working from home. From the initial literature review and learning of Python to the subsequent development of the research direction, I encountered numerous challenges. Nevertheless, with the support and assistance of others, these difficulties were ultimately overcome.

I would like to express my sincere gratitude to my thesis advisor, Professor Zhao Guoping. He never urged me to accelerate my progress; instead, whenever I encountered problems, he consistently provided patient guidance and valuable insights.

Finally, due to my limited professional expertise and the interdisciplinary nature of this research, there are inevitably shortcomings in this thesis despite repeated revisions and careful reflection throughout the writing process. I sincerely welcome any comments, criticisms, and suggestions from my professors.

References:

[1] Jia Chao, Su Wei. Analysis of Digital Transformation Pathways in the Manufacturing Sectors of the United States and Germany. Information Technology and Standardization, 2019(6):43-46.

[2] Zhang Shu. How Chinese Manufacturing Enterprises Can Transition to Industry 4.0. Mechanical Design and Manufacturing Engineering, 2014, 43(12): 1-5.

[3] Lu Yufeng. Research on Information-Based Methods for Quality Control in Foundry Processes [J]. China Manufacturing Informatization: Academic Edition, 2010, 39(12): 19-22.

[4] Huang Yueren. Development Trends of China's Machinery Manufacturing Industry in the Era of Industry 4.0 [J]. Modern Economic Information (10).

[5] Zhang Shu. Industry 4.0 and Smart Manufacturing [J]. Mechanical Design and Manufacturing Engineering, 2014(8):1-5.

[6] Wang Xiwen. Industry 4.0: Smart Industry. Internet of Things Technology, 2013, 3(12): 3-4.

[7] Yang Haicheng. Smart Manufacturing Engineering Drives Industrial Innovation and Development [J]. China Industrial Review, 2016, 000(010): pp. 78-82.

[8] Gong Tao, Zhao Henan. Construction of a New Data Acquisition System for Smart Factories [J]. Information Technology and Network Security, 2018, 37(3): 15-19.

[9] Ma Zhongya, Wei Zhixin. Reflections on Data Analysis in Manufacturing Informatization. Electronics Technology and Software Engineering, 2016(9):199-199.

[10] Tao Yong, Wang Tianmiao, Li Qiushí, et al. "Integration of Design, Manufacturing, and Service Across the Entire Manufacturing Lifecycle Based on 'Internet Plus' [J]. Science and Technology Review, 2016, Vol. 34; No. 490(04): 47-51.

[11] Yang Haicheng. A Comprehensive Understanding of "Internet Plus" and Vigorous Promotion of Smart Manufacturing [J]. Robotics Technology and Applications, 2015(04):30-32.

[12] Tu Cong. Research on Data Visualization Applications in the Era of Big Data [J]. Electronics Manufacturing, 2013, 47(5):491-498.

[13] Ren Yonggong, Yu Ge. An Image Post-processing Method Supporting Visual Data Mining [J]. Small and Micro Computer Systems, 2005(11):85-89.

[14] Yang Lu, Ge Wenqian. Design of a Python-Based Data Analysis and Visualization Platform for the Manufacturing Industry [J]. Research on Informatization, 2018, 44(05):60-65.

[15] Zhang Liangjun, Wang Lu, Tan Liyun, et al. Practical Python Data Analysis and Mining [M]. Beijing: China Machine Press, 2016.

[16] Zhang Shaofang, Zhao Lidong, Li Xianjun. A Data Acquisition and Management System for Foundry Production. Foundry Technology, 2015, 36(5):1290-1292.

[17] Zhang Fuqun, Song Xizhong, Liu Jinguang. A Data Acquisition and Intelligent Management System for Foundry Workshops Based on WSNs. Foundry Technology, 2015,

36(3):785-787.

[18] Wang Zhiping, Yang Jiao, Zhu Changsheng, Xiao Rongzhen, Feng Li, Liu Zhiyuan. Research and Development of a Foundry Process Database System. Foundry, 2010(8):810-813.

[19] Du Yuzhu, Tang Hongtao, Chen Tao. Construction and Application of an Integrated PLM/ERP/MES Information Platform for Intelligent Casting [J]. Foundry Engineering, 2019(1):35-39.

[20] How Far Are We from Intelligent Casting [J]. China Electromechanical Industry, 2016, 000(004):91-92.

[21] Liu Jinhai. Evaluation and Control of Metallurgical Quality of Cast Iron Melt [C]//2017 China Foundry Week. 0.

[22] Liu Xiangde, Jing Tao, Liu Baicheng. WEB-Based Simulation System of Casting Solidification Process [J]. Foundry, 2002, 51(8):498-501.

[23] Cheng Jingping. Key Technologies for a Real-Time Information-Based Quality Monitoring System in Foundries [J]. Journal of Huangshi Institute of Technology, 2007, 23(4):1-4.

[24] Xu Qingyan. Roadmap for Foundry Technology: Digital, Networked, and Intelligent Foundry [J]. Foundry, 2017, 066(012):1243-1250.

[25] Zhu Xianbin, Pu Yuchi, Hu Yixin. Research on the Design of a Collaborative Platform for PDM, ERP, and MES in Mechanical Factories [J]. Engineering Construction and Design, 2018.

[26] Jacek Kozłowski, Michał Jakimiuk, Michał Rogalewicz, et al. Analysis and Control of High-Pressure Die Casting Process Parameters with Use of Data Mining Tools [M] // Temporal Modelling of Customer Behaviour. 2019.

[27] Perzyk, M. “Statistical and Visualization Data Mining Tools for Foundry Production.” Archives of Foundry Engineering, 2007, pp. 111 – 116.

[28] Ghosh, Sushovan, and J. Maiti. “Data Mining Driven DMAIC Framework for Improving Foundry Quality—a Case Study.” Production Planning & Control, vol. 25, no. 6, 2014, pp. 478 – 493.

[29] Ravi, B., & Datta, G. (2020). Co-operative Virtual Foundry for Cost-Effective Casting Simulation.

[30] Rojek, G., et al. “Methods of Computational Intelligence in the Context of Quality Assurance in Foundry Products.” Archives of Foundry Engineering, vol. 16, no. 2, 2016, pp. 11 – 16.

[31] Perzyk, M., and J. Kozlowski. “Methodology of Fault Diagnosis in Ductile Iron Melting Process.” Archives of Foundry Engineering, vol. 16, no. 4, 2016, pp. 101 – 108.

[32] Perzyk, M., and A. Kochański. “Detection of Causes of Casting Defects Assisted by Artificial Neural Networks.” Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture, vol. 217, no. 9, 2003, pp. 1279 – 1284.

[33] Perzyk, M., and A. Rodziewicz. “Application of Special Cause Control Charts to the Green Sand Process.” Archives of Foundry Engineering, vol. 15, no. 4, 2015, pp. 55 – 60.

[34] Ignaszak, Z., and R. Sika. “Specificity of SPC Procedures Application in Foundry in Aspect of Data Acquisition and Data Exploration.” Archives of Foundry Engineering, vol. 12, no. 4, 2012, pp. 65 – 70.

[35] Zaludova, A. H. “Statistical Quality Control in the Foundry.” Production Engineer, vol. 43, no. 2, 1964, p. 88.

[36] Sika, Robert, and Paweł Popielarski. “Methodology Supporting Production Control in a Foundry Applying Modern DISAMATIC Molding Line.” Modern Technologies in Manufacturing (MTeM 2017 - AMaTUC), October 12 – 13, 2017, Cluj-Napoca, Romania, vol. 137, 2017, p. 5007.

[37] Sika, Robert, et al. “Methodology for Determining Key Casting Process Parameters on the DISA MATCH Automatic Molding Line That Affect the Formation of Alloy-Mold Contact Defects.” International Scientific-Technical Conference MANUFACTURING, 2019, pp. 416 – 433.

[38] Köksal, Gülser, et al. “Review: A Review of Data Mining Applications for

Quality Improvement in the Manufacturing Industry.” Expert Systems With Applications, vol. 38, no. 10, 2011, pp. 13448–13467.

[39] Zhong, Ray Y. “Analysis of RFID Datasets for Smart Manufacturing Shop Floors.” 2018 IEEE 15th International Conference on Networking, Sensing , and Control (ICNSC), 2018, pp. 1–4.

[40] Zawadzki, Przemysław, and Krzysztof Żywicki. “Smart Product Design and Production Control for Effective Mass Customization in the Industry 4.0 Concept.” Management and Production Engineering Review, vol. 7, no. 3, 2016, pp. 105–112.

[41] Trojanowska, Justyna, et al. “The Tool Supporting Decision Making Process in the Area of Job-Shop Scheduling.” World Conference on Information Systems and Technologies, vol. 571, 2017, pp. 490–498.

[42] Stukowski, Alexander. “Visualization and Analysis of Atomistic Simulation Data with OVITO—the Open Visualization Tool.” Modelling and Simulation in Materials Science and Engineering, vol. 18, no. 1, 2010, p. 15012.

[43] Hanwell, Marcus D., et al. “Avogadro: An Advanced Semantic Chemical Editor, Visualization, and Analysis Platform.” Journal of Cheminformatics, vol. 4, no. 1, 2012, pp. 17–19.

[44] Zhang Nan. Research on the Python Language and Its Application Fields [J]. Journal of Science and Technology Innovation, 2019(17).

[45] Zhao Guanghui. Case-Based Teaching of Python Programming for Interdisciplinary Integration in New Engineering Disciplines [J]. Computer Education, 2017(8).

[46] Yang Ying. Explorations in Python Teaching in the Context of Artificial Intelligence [J]. Information Systems Engineering, 2019, 000(004):175.

[47] Chen Guanghui. A Review of Python Applications and Development in the Internet of Things [J]. Information and Communications, 2019, 000(004):123-125.

[48] Li Fangyuan. Lecture 8 on Key Technologies in Smart Factories: Python Programming Applications in Smart Factories. Automation Review, 2019(5):54-55.

[49] Yang Wushuai, Wan Qiyuan, Sang Xianbo. Design and Implementation of an IoT Smart Device Management System Based on Python and the Django Framework [J]. China’s New Technologies and New Products, 2018, 000(011):13-15.

[50] Nie Jing. Analysis of the Application of Python in Big Data Mining and Analysis [J]. Journal of Guangxi Minzu University (Natural Science Edition), 2018, 024(001):76-79.

[51] Qin Ke, Liu Guisong. A Preliminary Exploration of University Teaching Reform Oriented toward Python Applications [J]. Computer Education, 2017(9).

[52] Song Yongsheng, Huang Rongmei, Wang Jun. Research on a Python-Based Data Analysis and Visualization Platform. Modern Information Technology, 2019, 3(21):7-9.

[53] Huang Qi. Data Visualization Methods and System Implementation Based on Python. Information and Computers, 2019(14):137-140.

[54] Cai Min. Analysis of Web Development Applications Using Python [J]. Wireless Internet Technology, 2019, 016(004):27-28.

[55] Chen Kaicong, Zhou Renjie, Zhu Zhisong. Industrial Control Data Exchange Implemented Using Python [J]. Machine Tools & Hydraulics, 2018(16):112-114.

[56] Xu Zhong, Lei Xizhang, Yang Guang, et al. Application of Python Data Analysis Tools in the Informatization Development of a CSP Plant [J]. Office Automation (Office Equipment and Supplies), 024(013):44-47.

[57] Jacek Kozłowski, Sika R, Filip Górski, et al. Modeling of Foundry Processes in the Era of Industry 4.0 [C]// Design, Simulation, Manufacturing: the Innovation Exchange. Springer, Cham, 2018.

[58] Song Tian, Huang Tianyu, Li Xin. Python: An Ideal Choice for Reforming Programming Course Instruction [J]. Teaching in Chinese Universities, 2016(2):42-47.

[59] “Current Status and Trends in the Development of Digital, Networked, and Intelligent Casting Technologies” Symposium Held in Shenyang [J]. Casting, 2018(1):96-97.